Name: Omar Osama Abdelmonem Amin Emam
Matric No: A25CS0017
SCSE2103 Data Structures and Algorithms - Section 03 - Session 2026/2027/1

Lab Exercise 1: Comprehensive DSA Debug and Code Completion

Chapters 1 - 4

1. Summary

The source file contains 10 tasks: 5 debugging tasks (1, 3, 5, 7, 9) and 5 completion tasks (2, 4, 6, 8, 10). All tasks were fixed or completed. The program compiles cleanly with g++ (-Wall -Wextra, no warnings) and the test bench produces the expected output.

2. Task Summary

No.	Type	Topic	What was done
1	Debug	Pointer to object	Replaced ptr.printStudent() with ptr->printStudent(); a pointer needs the arrow operator.
2	Complete	setGPA()	Accepts 0.0 to 4.0 inclusive and returns true; otherwise prints an error and returns false.
3	Debug	Array of objects	Loop condition i <= size read past the end of the array; changed to i < size.
4	Complete	Average GPA	Loop sums GPAs and returns total / size.
5	Debug	Binary file write	Opened with ios::out | ios::binary and replaced << with write(reinterpret_cast(...), sizeof(Student)).
6	Complete	Random access read	seekg(recordIndex * sizeof(Student)), then read() with reinterpret_cast; returns whether the read succeeded.
7	Debug	Recursive factorial	Added base case (n <= 1 returns 1) and changed the recursive call to n - 1.
8	Complete	Recursive search	Returns -1 when index reaches size, returns index on ID match, otherwise recurses on index + 1.
9	Debug	Step counting	Moved the inner loop update into the for header (j++) so termination is explicit. Steps = N^2.
10	Complete	Halving loop	for (i = n; i > 0; i /= 2) stepCount++; complexity is O(log N).

3. Key Code Changes

Task 5: binary file write (before / after)

```
// before
fstream outFile(filename, ios::out);
outFile << list[i];

// after
fstream outFile(filename, ios::out | ios::binary);
outFile.write(reinterpret_cast<const char*>(&list[i]), sizeof(Student));
```

Task 7: recursion (before / after)

```
// before (infinite recursion, no base case, n never decreases)
return n * recursiveFactorial(n);
```

```
// after
if (n <= 1) return 1;
return n * recursiveFactorial(n - 1);
```

Task 8: recursive search

```
if (currentIndex >= size) return -1;
if (list[currentIndex].getID() == targetID) return currentIndex;
return recursiveSearchID(list, size, targetID, currentIndex + 1);
```

Task 10: halving loop

```
for (int i = n; i > 0; i /= 2) {
    stepCount++;
}
// Complexity: O(log N)
```

4. Program Output

```
--- Testing ADT & Pointers ---
ID: 104 | Name: Nurul | GPA: 3.2

--- Student Roster ---
ID: 101 | Name: Ahmad | GPA: 3.75
ID: 102 | Name: Siti | GPA: 3.9
ID: 103 | Name: Chong | GPA: 3.5
Calculated Average GPA: 3.71667
Successfully saved records to binary file: students.dat
Successfully retrieved record at index 1:
ID: 102 | Name: Siti | GPA: 3.9
Recursive Factorial of 4: 24
Recursive Search for ID 102 Found at Index: 1

--- Algorithm Efficiency Analysis (N = 4) ---
Total Basic Operations Executed: 16 steps.
Theoretical Complexity: O(N^2)
Logarithmic Step Count for N=16: 5 steps.
```

5. Complexity Analysis

Task 9: the nested loops each run N times, so the inner statement executes N x N times (16 steps for N = 4), giving $O(N^2)$.

Task 10: i is halved each iteration (16, 8, 4, 2, 1), so the loop runs floor($\log_2$ N) + 1 times (5 steps for N = 16), giving O(log N).

6. Conclusion

All 10 tasks were completed and tested. The exercise covered pointers to objects, arrays of objects, binary file I/O with random access, recursion with correct base cases, and step counting for Big O analysis.